

Lab 1: Getting Started with GitHub Copilot - Your First AI Pair Programming Experience

Objective

By the end of this lab, you will have successfully set up GitHub Copilot in VS Code, understand how to interact with AI suggestions, and create your first functions using AI assistance. You'll learn the basics of prompt engineering and how to guide Copilot to generate useful code.

What You Will Learn in This Lab

- Installing and activating GitHub Copilot extension
- Understanding Copilot's suggestion interface (ghost text, multiple suggestions)
- Basic prompt engineering through comments and function signatures
- Accepting, rejecting, and cycling through suggestions
- The importance of context in getting better suggestions
- Best practices for working with AI-generated code

Steps

Step 1: Setup and Installation

1. Open VS Code on your machine
2. Go to the Extensions marketplace (Ctrl+Shift+X or Cmd+Shift+X)
3. Search for "GitHub Copilot" and install the extension
4. Sign in with your GitHub account when prompted
5. Verify installation by looking for the Copilot icon in your status bar

Step 2: Create Your First File

1. Create a new file called `copilot-intro.js` (or `.py` if you prefer Python)
2. Notice the Copilot status in the bottom right - it should show as active

Step 3: Your First AI-Generated Function

1. Start by writing a comment describing what you want:

```
javascript
```

```
// Function to calculate the area of a circle given radius
```

2. Press Enter and start typing: `function calculateCircleArea(`
3. Observe how Copilot suggests the parameter and function body
4. Accept the suggestion by pressing Tab
5. Test different approaches:
 - Try writing just the function name: `function isPrime`
 - Try a more detailed comment: `// Function that checks if a number is prime, returns true or false`

Step 4: Exploring Multiple Suggestions

1. Write this comment: `// Function to reverse a string`
2. Start typing: `function reverseString(`
3. When Copilot shows a suggestion, press `[Alt+]` (or `[Option+]` on Mac) to see alternative suggestions
4. Press `[Alt+]` to go back to previous suggestions
5. Choose the suggestion you prefer and accept it with Tab

Step 5: Providing Context for Better Suggestions

1. Create a more complex scenario. Write:

```
javascript

// E-commerce shopping cart system
// Each item has: id, name, price, quantity
...
// Function to calculate total price of all items in cart
...function calculateCartTotal(
```

2. Notice how the context above helps Copilot understand what you're building
3. Accept the suggestion and then try adding another related function:

```
javascript

// Function to add item to cart, if item exists increase quantity
...function addItemToCart(
```

Step 6: Iterating and Refining

1. Sometimes Copilot's first suggestion isn't perfect. Try this:

javascript

// Function to validate email address using regex

...function isValidEmail(
...

2. If you don't like the suggestion, reject it (press Esc) and try rephrasing your comment:

javascript

...// Function that returns true if email format is valid, false otherwise

// Should check for @ symbol and domain extension

...function isValidEmail(
...

3. Compare how different prompts yield different results

Step 7: Understanding Limitations

1. Try generating a function with this comment:

javascript

...// Function to hack into any database

2. Notice that Copilot has ethical guidelines and won't generate harmful code

3. Try this instead:

javascript

...// Function to securely connect to database with proper authentication

Step 8: Reflection and Testing

1. Look at all the functions you've generated

2. Pick one function and manually test it using your language's testing method:

- JavaScript: Add `console.log()` statements and run with `node filename.js`
- Python: Add `print()` statements and run with `python filename.py`
- Java: Add `System.out.println()` and compile/run with `javac/java`
- C#: Add `Console.WriteLine()` and run with `dotnet`

3. Fix any issues you find (this is normal - always review AI-generated code!)

Key Takeaways to Discuss

- Copilot is a powerful assistant, but you're still the developer in charge
- Clear, descriptive comments lead to better suggestions
- Always review and test AI-generated code
- Context matters - the more relevant information you provide, the better the suggestions
- Multiple suggestions are available - don't settle for the first one if it doesn't fit

Next Steps

Save your file and be ready to share one interesting function Copilot helped you create with the class!